

IACV 2025/26 PROJECT

Synthetic IPPE-Square Pose Estimation

Technical Project Report

Planar geometry, the two-pose ambiguity, and conditioning

Mathematical foundations, implementation, and synthetic experiments

Alessio Timofte
Anthony Iemmolo
Matteo Bianchi

September 2026

Contents

1	Overview and Notation	2
1.1	Notation	2
2	The Synthetic Pipeline	2
2.1	Step 1 — The 3D Marker Plane	2
2.2	Step 2 — The Square on the Plane	3
2.3	Step 3 — The Pinhole Camera	3
2.3.1	Geometric model	3
2.3.2	The intrinsic matrix $\mathbf{K}$	3
2.3.3	Normalized image coordinates	4
2.4	Step 4 — Ground-Truth Pose	4
2.4.1	The rotation group $\text{SO}(3)$	4
2.4.2	Representation 1: Euler angles	4
2.4.3	Facing the camera	5
2.4.4	Our concrete ground-truth pose	5
2.5	Step 5 — Forward Projection	5
2.6	Step 6 — IPPE-Square Pose Recovery	6
2.6.1	Input validation	6
2.6.2	From pixels to a homography	6
2.6.3	The two quantities IPPE needs	6
2.6.4	The IPPE construction	6
2.6.5	Translation, ranking, and the decision not to refine	7
2.6.6	Reference implementation	7
2.7	Step 7 — Validation Against OpenCV and the Ground Truth	7
3	Experiment 1 — The Two-Pose Ambiguity	8
3.1	The quantity we plot	9
3.2	Setup	9
3.3	Results	9
3.4	Discussion	10
3.4.1	Why the two poses coincide at 0°	10
3.4.2	Why the pose error is exactly twice the viewing angle	11
4	Experiment 2 — Conditioning and Monte Carlo	11
4.1	The Residual Jacobian	12
4.2	Singular Values and Observability	13
4.3	Units and the Dimensionless Jacobian	13
4.4	The Fronto-Parallel Case in Closed Form	13
4.5	Conditioning Along the Camera Arc	14
4.6	Monte Carlo Validation	16
4.6.1	The estimator	16
4.6.2	Procedure	17
4.6.3	Results	17
4.6.4	Interpretation	18
4.7	Discussion	19

1 Overview and Notation

We synthesize the image a calibrated pinhole camera would observe of a known planar square marker placed at a known pose. There is no detection or image-processing stage: all 3D–2D corner correspondences are generated mathematically.

Step 1. Generate a 3D plane (the marker plane).

Step 2. Define a square on that plane of side L .

Step 3. Define a camera with known intrinsics $\mathbf{K}$.

Step 4. Choose a known marker pose $(\mathbf{R}_{\text{gt}}, \mathbf{t}_{\text{gt}})$ relative to the camera.

Step 5. Project the four 3D corners into the image.

Step 6. Recover both planar IPPE-square pose candidates from the correspondences.

Step 7. Compare the custom primary candidate to OpenCV’s `SOLVEPNP_IPPE_SQUARE` in the noiseless case.

Section 2 follows these seven steps in order. Two experiments then build on the finished pipeline: Section 3 sweeps the camera viewpoint and studies the two planar candidates, and Section 4 analyses the reprojection Jacobian and compares its uncertainty prediction with a Monte Carlo simulation.

1.1 Notation

- Vectors in bold lowercase: $\mathbf{t}$. Matrices in bold uppercase: $\mathbf{R}, \mathbf{K}$.
- Points carry a subscript naming their frame: $\mathbf{P}_{\text{mk}}$ is in the marker frame, $\mathbf{P}_{\text{cam}}$ in the camera frame.
- Pixel coordinates: $\mathbf{p} = (u, v)^\top$.
- Coordinate frames are right-handed: in the camera frame, X points right, Y points down, Z points into the scene.

2 The Synthetic Pipeline

Steps 1–5 build a scene and the image a calibrated camera would record of it; steps 6–7 recover the pose from that image and check the result. Each subsection below corresponds to one step of the list in Section 1 and to one stage of `pipeline.py`, which prints every intermediate quantity when run.

2.1 Step 1 — The 3D Marker Plane

The marker is a flat object, so the four corners are coplanar. We attach a coordinate frame to the marker (the *marker frame*) with the plane chosen as $Z = 0$:

$$\text{marker plane} = \{ (X, Y, 0) : X, Y \in \mathbb{R} \}. \quad (1)$$

The origin is placed at the geometric center of the future square. The X axis points right, Y points up (in the plane), and Z is the outward normal to the marker surface.

Why $Z = 0$ matters. A general projection of a 3D point requires the full 3×3 rotation matrix. If the third coordinate is identically zero, the third column of $\mathbf{R}$ multiplies 0 and drops

out:

$$\mathbf{R} \begin{pmatrix} X \\ Y \\ 0 \end{pmatrix} = X\mathbf{r}_1 + Y\mathbf{r}_2, \quad (2)$$

where $\mathbf{r}_1, \mathbf{r}_2$ are the first two columns of $\mathbf{R}$. This collapse is the structural reason a single 3×3 matrix — a *homography* — captures all the geometry of a planar marker, which is what later pose-estimation steps will exploit.

2.2 Step 2 — The Square on the Plane

On the plane $Z = 0$ we place a square of side L , centered at the marker origin and axis-aligned to the marker frame. The four corner coordinates, in the order used by OpenCV’s `SOLVEPNP_IPPE_SQUARE`, are

$$\mathbf{P}_{\text{mk}}^{(0)} = \begin{pmatrix} -L/2 \\ +L/2 \\ 0 \end{pmatrix}, \mathbf{P}_{\text{mk}}^{(1)} = \begin{pmatrix} +L/2 \\ +L/2 \\ 0 \end{pmatrix}, \mathbf{P}_{\text{mk}}^{(2)} = \begin{pmatrix} +L/2 \\ -L/2 \\ 0 \end{pmatrix}, \mathbf{P}_{\text{mk}}^{(3)} = \begin{pmatrix} -L/2 \\ -L/2 \\ 0 \end{pmatrix}. \quad (3)$$

The parenthesized superscript is the corner index; the subscript “mk” indicates the marker frame.

Corner ordering matters. The order of the 3D list must match the order of the 2D observations, otherwise the correspondences are scrambled and the algorithm finds the wrong pose (or fails). The order above (clockwise starting at top-left, when the marker frame is drawn with X right and Y up and seen from $+Z$) is the convention shared by OpenCV’s `ArUco` detector and `SOLVEPNP_IPPE_SQUARE`. The sign of the shoelace sum over the four corners is negative, which is the quick numerical check for this orientation.

2.3 Step 3 — The Pinhole Camera

2.3.1 Geometric model

A pinhole camera maps points in the camera frame to points on a 2D image. The camera origin is at the world origin; the image plane sits at $Z = f$, where f is the focal length. By similar triangles, a 3D point $\mathbf{P}_{\text{cam}} = (X, Y, Z)^\top$ projects to

$$x = f \cdot \frac{X}{Z}, \quad y = f \cdot \frac{Y}{Z}. \quad (4)$$

The division by Z is the *perspective divide*: it is the only non-linearity in the model and is responsible for foreshortening (farther objects look smaller).

2.3.2 The intrinsic matrix $\mathbf{K}$

Real images are measured in pixels. The focal length is encoded as f_x, f_y (in pixels, possibly different along the two axes if the pixel elements are not square), and the optical axis hits the image at the *principal point* (c_x, c_y) . The pixel coordinates are

$$u = f_x \cdot \frac{X}{Z} + c_x, \quad v = f_y \cdot \frac{Y}{Z} + c_y. \quad (5)$$

Collected into a matrix:

$$\mathbf{K} = \begin{pmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix} \quad (6)$$

so that the projection has the compact homogeneous form

$$\tilde{\mathbf{p}} = \mathbf{K} \mathbf{P}_{\text{cam}}, \quad \mathbf{p} = \begin{pmatrix} \tilde{p}_1 / \tilde{p}_3 \\ \tilde{p}_2 / \tilde{p}_3 \end{pmatrix}. \quad (7)$$

The first is a linear matrix product, the second is the perspective divide.

2.3.3 Normalized image coordinates

Apply $\mathbf{K}^{-1}$ to a pixel $\mathbf{p}$:

$$\begin{pmatrix} x_n \\ y_n \\ 1 \end{pmatrix} = \mathbf{K}^{-1} \begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = \begin{pmatrix} (u - c_x)/f_x \\ (v - c_y)/f_y \\ 1 \end{pmatrix}. \quad (8)$$

The pair (x_n, y_n) are the *normalized image coordinates*: the location where the ray through pixel $\mathbf{p}$ crosses the plane $Z = 1$ in camera space. They are dimensionless (no pixels, no focal length) and encode pure ray directions. Pose-estimation algorithms work in this representation so that the camera-specific quantities $\mathbf{K}$ do not contaminate the geometric reasoning.

2.4 Step 4 — Ground-Truth Pose

A point in the marker frame is brought into the camera frame by a rigid-body transform:

$$\boxed{\mathbf{P}_{\text{cam}} = \mathbf{R} \mathbf{P}_{\text{mk}} + \mathbf{t}} \quad (9)$$

with $\mathbf{R} \in \text{SO}(3)$ and $\mathbf{t} \in \mathbb{R}^3$. The pair $(\mathbf{R}, \mathbf{t})$ is an element of the Special Euclidean group $\text{SE}(3)$, and is what we mean by *pose*.

2.4.1 The rotation group $\text{SO}(3)$

A 3×3 matrix $\mathbf{R}$ is a rotation if and only if

$$\mathbf{R}^\top \mathbf{R} = \mathbf{I}, \quad \det(\mathbf{R}) = +1. \quad (10)$$

The first condition (orthonormal columns) preserves lengths and angles; the second excludes reflections, which would flip a right-handed frame into a left-handed one.

Degrees of freedom. A 3×3 matrix has 9 entries. The orthogonality condition $\mathbf{R}^\top \mathbf{R} = \mathbf{I}$ imposes 6 independent constraints. That leaves **3 DOF** — the minimum number of parameters needed to describe a rotation in 3D.

2.4.2 Representation 1: Euler angles

Decompose any rotation as three elementary rotations around the coordinate axes. The code uses SciPy’s lower-case *extrinsic xyz* convention: the rotations are applied about fixed camera axes, and the resulting active matrix is

$$\mathbf{R}(\alpha, \beta, \gamma) = \mathbf{R}_z(\gamma) \mathbf{R}_y(\beta) \mathbf{R}_x(\alpha). \quad (11)$$

where each $\mathbf{R}_\bullet(\cdot)$ is a 3×3 planar rotation, e.g.

$$\mathbf{R}_x(\alpha) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos \alpha & -\sin \alpha \\ 0 & \sin \alpha & \cos \alpha \end{pmatrix}. \quad (12)$$

Pros: human-intuitive — $(15^\circ, 10^\circ, 5^\circ)$ is easy to picture.

Cons: suffer from *gimbal lock* (a configuration where two axes align and one DOF is lost) and from non-unique decompositions. We use Euler angles only to *specify* the ground-truth pose conveniently; we never use them for optimization.

2.4.3 Facing the camera

One detail is easy to miss. If $\mathbf{R} = \mathbf{I}$, the marker axes coincide with the camera axes, so the marker normal $+Z$ points *into* the scene, away from the camera: we would be looking at the back of the marker. A marker that faces the camera, as a detected ArUco marker always does, is therefore near a 180° rotation, not near the identity. We build the pose as

$$\mathbf{R}_{\text{gt}} = \mathbf{R}(\alpha, \beta, \gamma) \mathbf{F}, \quad \mathbf{F} = \text{diag}(1, -1, -1), \quad (13)$$

where $\mathbf{F}$ is a 180° rotation about X (a proper rotation: $\det \mathbf{F} = +1$). Because $\mathbf{F}$ is applied first, in the marker frame, the Euler angles keep their intended meaning: a tilt away from the fronto-parallel view.

2.4.4 Our concrete ground-truth pose

In the code we choose

$$\text{Euler (xyz, deg)} : \quad (\alpha, \beta, \gamma) = (15, 10, 5) \quad (14)$$

$$\mathbf{t}_{\text{gt}} : \quad (0.05, 0.02, 0.50)^\top \text{ m} \quad (15)$$

which, through (13), yields

$$\mathbf{R}_{\text{gt}} \approx \begin{pmatrix} +0.9811 & +0.0394 & -0.1897 \\ +0.0858 & -0.9662 & +0.2432 \\ -0.1736 & -0.2549 & -0.9513 \end{pmatrix}.$$

The Euler part alone rotates by $\approx 18.33^\circ$, which is *not* $15^\circ + 10^\circ + 5^\circ$ because Euler rotations do not compose by simple addition. The third column of $\mathbf{R}_{\text{gt}}$ is the marker normal expressed in the camera frame, $(-0.190, 0.243, -0.951)^\top$; its negative Z component confirms that the normal points back towards the camera.

Viewing angle. Throughout this report the *viewing angle* is the angle between the marker normal and the line from the marker centre to the camera centre: 0° is fronto-parallel, and a value above 90° would mean we are looking at the back of the marker. For the pose above it is 17.26° , so the baseline scene is a mildly oblique but ordinary view, directly comparable with the sweep of Section 3.

2.5 Step 5 — Forward Projection

Steps 1–4 fix all the inputs. Step 5 chains them into the observed image points. For each marker corner $\mathbf{P}_{\text{mk}}^{(i)}$:

$$\boxed{\mathbf{p}^{(i)} = \pi\left(\mathbf{K}(\mathbf{R}_{\text{gt}}\mathbf{P}_{\text{mk}}^{(i)} + \mathbf{t}_{\text{gt}})\right)} \quad (16)$$

where the projection $\pi : \mathbb{R}^3 \rightarrow \mathbb{R}^2$ is the perspective divide

$$\pi \begin{pmatrix} \tilde{x} \\ \tilde{y} \\ \tilde{z} \end{pmatrix} = \begin{pmatrix} \tilde{x}/\tilde{z} \\ \tilde{y}/\tilde{z} \end{pmatrix}. \quad (17)$$

In the code, Step 5 is split in two sub-steps:

- (a) **Rigid transform.** $\mathbf{P}_{\text{cam}}^{(i)} = \mathbf{R}_{\text{gt}}\mathbf{P}_{\text{mk}}^{(i)} + \mathbf{t}_{\text{gt}}$. Brings each marker corner from the marker frame into the camera frame.
- (b) **Pinhole projection.** $\mathbf{p}^{(i)} = \pi(\mathbf{K}\mathbf{P}_{\text{cam}}^{(i)})$. Maps the camera-frame point to pixel coordinates via (5).

Possible sanity checks at this point:

- All $\mathbf{P}_{\text{cam}}^{(i)}$ have $Z > 0$ (in front of the camera).
- All $\mathbf{p}^{(i)}$ fall inside $[0, W) \times [0, H)$ (visible in the image, W as width, H as height).
- The four $\mathbf{p}^{(i)}$ form a *convex quadrilateral* that looks like a perspective-warped square.

2.6 Step 6 — IPPE-Square Pose Recovery

The marker is planar, so its normalized-image projection is a homography

$$\begin{pmatrix} x_n \\ y_n \\ 1 \end{pmatrix} \sim \mathbf{H} \begin{pmatrix} X \\ Y \\ 1 \end{pmatrix}. \quad (18)$$

Our solver is written from scratch in `ippe_square.py` and never calls OpenCV. It follows the planar-homography formulation of IPPE, and this section describes what the implementation actually computes, function by function.

2.6.1 Input validation

IPPE_SQUARE is not a generic coplanar solver: it assumes the four object points are the centred square of Step 2, in that exact order. Feeding it a different layout still produces a pose, only a meaningless one. The solver therefore first checks that the points are finite, lie on $Z = 0$, and match the canonical square of side $L = \|\mathbf{P}_{\text{mk}}^{(1)} - \mathbf{P}_{\text{mk}}^{(0)}\|$, and refuses anything else.

2.6.2 From pixels to a homography

Pixels are first mapped to normalized coordinates with $\mathbf{K}^{-1}$, so the intrinsics disappear from everything that follows. The homography is then estimated by the standard DLT: each correspondence $((X, Y), (x_n, y_n))$ contributes two rows to a 8×9 design matrix, and $\mathbf{H}$ is read off the right singular vector of its smallest singular value. The homogeneous scale is fixed by dividing by h_{33} , which is safe here because the square is centred at the origin and lies in front of the camera.

2.6.3 The two quantities IPPE needs

IPPE is a *local* method: it uses the homography only through its value and its first derivative at one point, here the square centre $(X, Y) = (0, 0)$. Since the centre is the origin, its normalized projection is read directly off the last column,

$$\mathbf{v} = (h_{13}, h_{23})^\top, \quad (19)$$

and the derivative follows from the quotient rule applied to $x_n(X, Y) = (h_{11}X + h_{12}Y + h_{13}) / (h_{31}X + h_{32}Y + 1)$, evaluated at the origin:

$$\mathbf{J}_{\mathbf{H}} = \begin{pmatrix} h_{11} - h_{13}h_{31} & h_{12} - h_{13}h_{32} \\ h_{21} - h_{23}h_{31} & h_{22} - h_{23}h_{32} \end{pmatrix}. \quad (20)$$

2.6.4 The IPPE construction

Let $\mathbf{R}_{\mathbf{v}}$ be the smallest rotation taking $\mathbf{e}_3$ onto $(\mathbf{v}, 1)^\top$, computed in closed form by Rodrigues' formula. Working in the frame it defines removes the effect of the marker being off the optical axis. Writing

$$\mathbf{B} = \begin{pmatrix} 1 & 0 & -v_1 \\ 0 & 1 & -v_2 \end{pmatrix} [\mathbf{R}_{\mathbf{v}}]_{:,1:2}, \quad \mathbf{A} = \mathbf{B}^{-1} \mathbf{J}_{\mathbf{H}}, \quad (21)$$

the matrix $\mathbf{A}$ is, up to scale, the upper-left 2×2 block of the rotation we are looking for. Its largest singular value γ supplies that scale, and we set $\mathbf{R}_{2 \times 2} = \mathbf{A}/\gamma$.

The rotation is then completed column by column. Orthonormality forces

$$\mathbf{b} \mathbf{b}^\top = \mathbf{I} - \mathbf{R}_{2 \times 2}^\top \mathbf{R}_{2 \times 2}, \quad (22)$$

whose right-hand side is positive semidefinite of rank one; $\mathbf{b}$ is recovered from its largest eigenpair. The remaining entries come from the cross product of the first two columns, which must be the third. Both signs of $\mathbf{b}$ satisfy (22), and this is precisely where the planar ambiguity enters: the two candidates are

$$\mathbf{R}^\pm = \mathbf{R}_v \begin{pmatrix} \mathbf{R}_{2 \times 2} & \pm \mathbf{c} \\ \pm \mathbf{b}^\top & a \end{pmatrix}, \quad (23)$$

with $(\mathbf{c}, a)$ obtained from that cross product. At a fronto-parallel view the right-hand side of (22) vanishes, so $\mathbf{b} = \mathbf{0}$ and the two candidates collapse into one; Section 3 returns to this.

2.6.5 Translation, ranking, and the decision not to refine

With a rotation fixed, translation is linear. Each corner requires its camera ray $\mathbf{q}^{(i)} = (x_n^{(i)}, y_n^{(i)}, 1)^\top$ to be parallel to the transformed point, that is

$$\mathbf{q}^{(i)} \times (\mathbf{R} \mathbf{P}_{\text{mk}}^{(i)} + \mathbf{t}) = \mathbf{0} \iff [\mathbf{q}^{(i)}]_\times \mathbf{t} = -[\mathbf{q}^{(i)}]_\times \mathbf{R} \mathbf{P}_{\text{mk}}^{(i)}. \quad (24)$$

Only two of the three rows per corner are independent: writing $\mathbf{P} = \mathbf{R} \mathbf{P}_{\text{mk}}^{(i)} + \mathbf{t}$, the first two rows read $P_2 - y_n P_3 = 0$ and $P_1 - x_n P_3 = 0$, while the third, $x_n P_2 - y_n P_1 = 0$, is x_n times the first minus y_n times the second. We keep the first two, so stacking the four corners gives an 8×3 system, solved in the least-squares sense. This is the same system OpenCV solves; Section 2.7 explains why the choice matters.

Both candidates are then reprojected, ranked by pixel RMSE, and returned together. We deliberately apply *no* nonlinear refinement: an unconstrained optimization started from the two candidates can pull both to the same pose, which would erase the very ambiguity Experiment 1 sets out to measure.

2.6.6 Reference implementation

The result is compared, in the same noiseless scene, with OpenCV's `solvePnPGeneric` called with the `SOLVEPNP_IPPE_SQUARE` flag, using the same canonical point order. OpenCV serves only as an external reference: no part of the solver depends on it.

2.7 Step 7 — Validation Against OpenCV and the Ground Truth

Table 1 collects the four poses returned for the pose of Section 2.4.4. The primary candidate reproduces the ground truth to machine precision, and the per-corner residuals are of order 10^{-13} px, so the solver is exact in the noiseless case rather than merely accurate.

The candidate-1 values should not be compared with each other. The exact error is zero, since the pose fits the data perfectly, so what is printed is pure floating-point rounding. A double carries about 16 significant digits, and with pixel coordinates near 500 px the smallest representable step is about $500 \times 2.2 \cdot 10^{-16} \approx 10^{-13}$ px. Both $6.4 \cdot 10^{-14}$ and $2.2 \cdot 10^{-14}$ lie below that floor, so both are zero for every practical purpose; which one comes out larger depends only on the order of the floating-point operations inside each solver. The same absolute difference, about $2 \cdot 10^{-14}$ px, separates the two candidate-2 values, where it is simply invisible next to 3.3 px.

Candidate	Solver	RMSE (px)	Rot. error (°)	Transl. error (mm)
1	custom	$6.4 \cdot 10^{-14}$	0	$1.1 \cdot 10^{-13}$
2	custom	3.33825	34.521	3.101
1	OpenCV	$2.2 \cdot 10^{-14}$	0	$1.1 \cdot 10^{-13}$
2	OpenCV	3.33825	34.521	3.101

Table 1: Baseline scene: both IPPE-square candidates from the custom solver and from OpenCV, each compared with the ground-truth pose.

Two observations are worth making. First, the two solvers agree on *both* candidates: to 10^{-13} mm on the primary pose, and to every printed digit on the mirror one, which validates the custom implementation. Second, the mirror candidate is 34.52° away from the ground truth, that is exactly twice the 17.26° viewing angle of the scene; Section 3 shows this is not a coincidence.

A limitation of OpenCV’s IPPE at 180° . While comparing the two solvers along the camera arc of Section 3, OpenCV returned wrong poses: about 113 px of reprojection error on data our solver fits to 10^{-13} px. The data were not at fault, since OpenCV’s own `SOLVEPNP_SQPNP` solves them to 10^{-14} px, and the generic `SOLVEPNP_IPPE` flag fails in the same way as `SOLVEPNP_IPPE_SQUARE`. The translation it returned was correct and only the rotation was wrong, which points at the output stage rather than at the IPPE construction. The culprit is the private routine that converts the rotation matrix into a rotation vector. It uses the textbook formula

$$\omega = \arccos\left(\frac{\text{tr} \mathbf{R} - 1}{2}\right), \quad \omega = \frac{\omega}{2 \sin \omega} \begin{pmatrix} R_{32} - R_{23} \\ R_{13} - R_{31} \\ R_{21} - R_{12} \end{pmatrix}, \quad (25)$$

with no special case near $\omega = 180^\circ$, where both $\sin \omega$ and the antisymmetric part of $\mathbf{R}$ vanish and the axis becomes $0/0$. A marker facing the camera is close to a half turn, and every pose on our arc is *exactly* one, so it is lost entirely; OpenCV’s own `cv2.Rodrigues`, which handles this case, recovers the same rotations without error. The degradation is gradual as the angle approaches 180° :

Euler angles in the configuration	Rotation angle	OpenCV best RMSE (px)
$(15^\circ, 10^\circ, 5^\circ)$, the baseline	165.5°	$2 \cdot 10^{-14}$
$(5^\circ, 3^\circ, 1^\circ)$	175.0°	$4 \cdot 10^{-13}$
$(1^\circ, 0^\circ, 0^\circ)$	179.0°	$6 \cdot 10^{-12}$
$(0^\circ, 0^\circ, 0^\circ)$	180.0°	1.21

Table 2: OpenCV’s IPPE-square accuracy as the ground-truth rotation approaches a half turn. With the facing flip of (13), zero Euler angles give exactly 180° .

Two consequences follow. The baseline, at 165.5° , is unaffected, so the validation above stands. But the natural test of setting all three Euler angles to zero would make OpenCV fail while our solver remains exact, and it is one more reason Experiment 1 relies on our implementation: with OpenCV’s IPPE the sweep could not have been run at all.

3 Experiment 1 — The Two-Pose Ambiguity

A planar marker admits two poses that explain the same image almost equally well. This experiment measures *how* equally, as a function of the viewing angle: when the two candidates are easy to tell apart, and when they are not.

3.1 The quantity we plot

Everything below is measured with the reprojection error, so it is worth stating exactly what that number is. Given an estimated pose $(\mathbf{R}, \mathbf{t})$, we project the four known corners again and compare them with the observed pixels. Each corner contributes a two-component residual,

$$\mathbf{e}^{(i)} = \underbrace{\pi(\mathbf{K}(\mathbf{R} \mathbf{P}_{\text{mk}}^{(i)} + \mathbf{t}))}_{\text{where the pose says corner } i \text{ should be}} - \underbrace{\mathbf{p}^{(i)}}_{\text{where it is observed}} = \begin{pmatrix} \Delta u^{(i)} \\ \Delta v^{(i)} \end{pmatrix}, \quad (26)$$

so four corners give $2N = 8$ scalar residuals in total. Summing their squares gives the reprojection cost itself,

$$E(\mathbf{R}, \mathbf{t}) = \sum_{i=1}^N \|\mathbf{e}^{(i)}\|^2 = \sum_{i=1}^N [(\Delta u^{(i)})^2 + (\Delta v^{(i)})^2], \quad (27)$$

which is the quantity Section 4 minimizes. What we plot is its root mean square,

$$\boxed{\text{RMSE} = \sqrt{\frac{E(\mathbf{R}, \mathbf{t})}{2N}} = \sqrt{\frac{1}{2N} \sum_{i=1}^N [(\Delta u^{(i)})^2 + (\Delta v^{(i)})^2]}, \quad N = 4.} \quad (28)$$

Rationale behind RMSE: Both indexes carry the same information, since $\text{RMSE} = \sqrt{E/2N}$ is a monotone function of E , but the former improves significantly the readability and has been preferred in the literature. This is the convention OpenCV reports, so our numbers and its own are directly comparable.

The normalization factor of $2N$, the number of scalar residuals, removes the dependency from the number of corners N and taking the square root provides a result in *pixels*, that also makes it directly comparable with the corner noise σ_{px} .

3.2 Setup

The camera centre moves on an arc of fixed radius $r = 0.5$ m around the marker, always aimed at the marker centre, sampling one viewpoint per degree from 0° (fronto-parallel) to 89° (almost edge-on). No noise is added: the image points are the exact projections. At each viewpoint we solve with our own IPPE implementation of Section 2.6, keep *both* candidates, and record the reprojection RMSE and the pose error of each. Using our solver rather than OpenCV is what makes the experiment possible at all: both candidates are returned unrefined, so neither is silently discarded.

One arc, not a hemisphere. The camera path has a single free parameter. Both the azimuth, fixed at 0° , and the radius, fixed at 0.5 m, are held constant, so the camera slides along one meridian rather than covering the whole front hemisphere. The square is symmetric under 90° in-plane rotations, so other azimuths mainly relabel the corners and give equivalent curves; the radius, by contrast, changes the apparent size of the marker and therefore the scale of the reprojection errors. The results below should be read as belonging to this configuration.

3.3 Results

Figure 1 shows the two curves. The true pose (candidate 1) sits at $\approx 10^{-13}$ px everywhere, as it must without noise. The mirror pose (candidate 2) starts at zero and grows monotonically to 12.15 px.

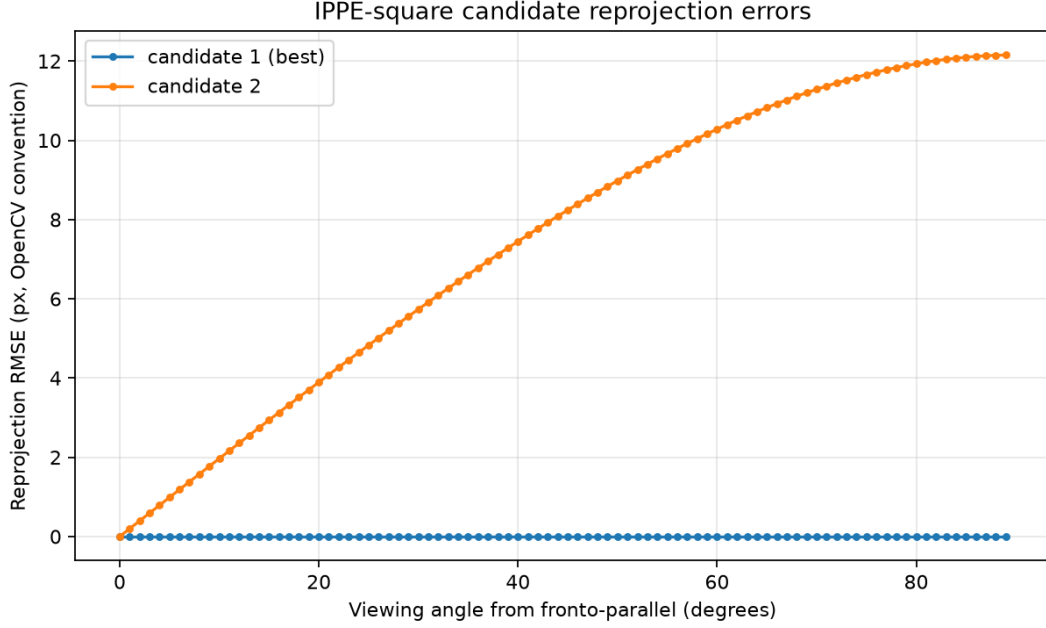


Figure 1: Reprojection RMSE of the two IPPE-square candidates against the viewing angle, one sample per degree.

Viewing angle (°)	Mirror RMSE (px)	Mirror rot. error (°)
0	0.000	0
1	0.197	2
5	0.986	10
15	2.939	30
30	5.736	60
45	8.240	90
60	10.281	120
75	11.659	150
89	12.153	178

Table 3: How the mirror candidate separates from the true pose, in the image and in pose space. The rotation error is exactly twice the viewing angle at every sampled point.

3.4 Discussion

3.4.1 Why the two poses coincide at 0°

The collapse is visible directly in the algebra of Section 2.6. The two candidates differ only through the vector $\mathbf{b}$, recovered from (22),

$$\mathbf{b}\mathbf{b}^\top = \mathbf{I} - \mathbf{R}_{2\times 2}^\top \mathbf{R}_{2\times 2}, \quad (29)$$

so everything hinges on how far $\mathbf{R}_{2\times 2}$ is from being orthogonal. At a fronto-parallel view the marker plane is parallel to the image plane, so the map from the plane to the image is a pure similarity: a rotation and a uniform scale, with no foreshortening in any direction. Its local derivative $\mathbf{A}$ is then exactly a scaled rotation, and dividing by its largest singular value γ leaves $\mathbf{R}_{2\times 2}$ orthogonal. The right-hand side vanishes, hence $\mathbf{b} = \mathbf{0}$.

The third column follows: with $\mathbf{b} = \mathbf{0}$ the cross product of the first two columns is $(0, 0, \pm 1)^\top$, so $\mathbf{c} = \mathbf{0}$ as well. Both sign choices produce the *same* matrix, and the solver returns one pose twice.

Away from 0° the plane is foreshortened along one direction only, so $\mathbf{A}$ is no longer conformal,

its two singular values separate, and $\mathbf{b}$ becomes non-zero. Measuring it along the sweep gives a clean identity,

$$\|\mathbf{b}\| = \sin \theta, \quad (30)$$

which we verified to machine precision at every sampled angle ($\theta = 1^\circ$: 0.017452; $\theta = 20^\circ$: 0.342020; $\theta = 45^\circ$: 0.707107). So $\|\mathbf{b}\|$ is exactly the *amount of ambiguity* in the configuration: zero head-on, maximal edge-on, and growing as the sine of the tilt. The degeneracy at 0° is not a numerical accident but a property of the geometry.

3.4.2 Why the pose error is exactly twice the viewing angle

Column three of Table 3 reads 2θ at every sampled angle, with no rounding error. The reason is that IPPE’s two solutions place the marker normal symmetrically about the line of sight.

Let $\mathbf{d}$ be the unit vector from the marker centre to the camera centre. By definition of the viewing angle, the true normal $\mathbf{n}_1$ makes an angle θ with $\mathbf{d}$. Both candidates reproduce the same local homography, and the only other plane orientation that does so is the one obtained by tilting the normal by θ on the *other* side of $\mathbf{d}$. The two normals are therefore coplanar with $\mathbf{d}$ and separated by

$$\angle(\mathbf{n}_1, \mathbf{n}_2) = 2\theta. \quad (31)$$

That accounts for the normals, but a rotation has three degrees of freedom and the normal fixes only two: in principle the two candidates could also differ by an in-plane spin, which would make the geodesic distance *larger* than 2θ . They do not. Computing the relative rotation $\mathbf{R}_1^\top \mathbf{R}_2$ along the sweep shows that its axis is, to machine precision, the marker’s Y axis — the in-plane axis orthogonal to the tilt direction, the azimuth being 0° — and its angle is exactly 2θ :

$$\mathbf{R}_1^\top \mathbf{R}_2 = \exp([2\theta \mathbf{u}]_\times), \quad \mathbf{u} \perp \mathbf{n}_1, \mathbf{u} \text{ in the marker plane.} \quad (32)$$

In other words the mirror candidate is the true pose *flipped about an in-plane axis*, with no extra spin: the whole discrepancy is a re-orientation of the plane, which is why the geodesic error and the angle between the normals agree. The consequence is uncomfortable, and is the point of the experiment: at a modest $\theta = 5^\circ$ the wrong candidate is already 10° away in 3D, while its image differs by about 1 px.

The image is far less sensitive than the pose. At 5° the two poses are 10° apart yet their images differ by about 1 px. This gap between 3D separation and image separation is the practical form of the ambiguity: the pixels simply do not carry enough evidence to decide.

Separability depends on the noise. With image noise of standard deviation σ_{px} , the two candidates are distinguishable only where the mirror RMSE is comfortably above σ_{px} . For the $\sigma_{px} = 0.5$ px used in Experiment 2, that threshold falls at around $2\text{--}3^\circ$ for this configuration. Both the distance and the marker size enter here through the apparent size of the square, so the threshold is a property of the configuration and not of the method.

The curve saturates. Beyond 75° the mirror RMSE gains only half a pixel, and by 89° the marker is under three pixels wide. Extreme obliquity separates the candidates no further while making every other aspect of the problem worse.

4 Experiment 2 — Conditioning and Monte Carlo

The second experiment treats pose estimation from the marker as the least-squares problem of minimizing the reprojection cost $E(\mathbf{R}, \mathbf{t})$ of (27). It addresses two questions. First, how

stable is that minimization with respect to noise on the four corners, as measured by the conditioning of its Jacobian at every viewing angle, and which directions of the pose are poorly observable? Second, does the uncertainty predicted from the Jacobian agree with a Monte Carlo simulation in which noise is added to the corners? Section 4.1 derives the Jacobian and Section 4.2 relates its singular values to the uncertainty of the estimate. Section 4.3 fixes the units in which conditioning is measured, Section 4.4 solves the fronto-parallel case in closed form, and Section 4.5 reports the conditioning along the camera arc. Section 4.6 presents the Monte Carlo validation and Section 4.7 draws the conclusions.

4.1 The Residual Jacobian

Let $\mathbf{e} \in \mathbb{R}^8$ stack the residuals (26) of the four corners, so that $E = \|\mathbf{e}\|^2$. Four corners provide eight equations for the six unknowns of the pose, so the problem is over-determined by two and noise cannot in general be fitted exactly.

Local parametrization. Translation is perturbed additively, $\mathbf{t} \mapsto \mathbf{t} + \delta\mathbf{t}$. Rotation is perturbed through a local rotation vector applied on the right,

$$\mathbf{R}(\delta\boldsymbol{\omega}) = \mathbf{R} \exp([\delta\boldsymbol{\omega}]_{\times}), \quad \exp([\delta\boldsymbol{\omega}]_{\times}) = \mathbf{I} + [\delta\boldsymbol{\omega}]_{\times} + O(\|\delta\boldsymbol{\omega}\|^2), \quad (33)$$

so that $\delta\boldsymbol{\omega}$ is expressed in the marker frame. The six pose parameters are ordered as

$$\delta\boldsymbol{\theta} = (\delta t_x, \delta t_y, \delta t_z, \delta\omega_x, \delta\omega_y, \delta\omega_z)^\top, \quad (34)$$

with translations in metres and rotations in radians. Euler angles are avoided: they introduce singularities and make the Jacobian depend on an arbitrary axis order.

Derivation. The residual of corner i depends on the pose only through the camera-frame point $\mathbf{P}_{\text{cam}}^{(i)} = \mathbf{R} \mathbf{P}_{\text{mk}}^{(i)} + \mathbf{t}$, so by the chain rule its Jacobian is the product of two factors. Substituting (33) and using $[\delta\boldsymbol{\omega}]_{\times} \mathbf{P}_{\text{mk}} = \delta\boldsymbol{\omega} \times \mathbf{P}_{\text{mk}} = -[\mathbf{P}_{\text{mk}}]_{\times} \delta\boldsymbol{\omega}$ gives the first,

$$\frac{\partial \mathbf{P}_{\text{cam}}^{(i)}}{\partial \delta\boldsymbol{\theta}} = \begin{bmatrix} \mathbf{I}_3 & -\mathbf{R} [\mathbf{P}_{\text{mk}}^{(i)}]_{\times} \end{bmatrix} \quad (3 \times 6). \quad (35)$$

Differentiating the pixel projection (5) with respect to $\mathbf{P}_{\text{cam}} = (X, Y, Z)^\top$ gives the second,

$$\mathbf{D}_i = \begin{bmatrix} f_x/Z & 0 & -f_x X/Z^2 \\ 0 & f_y/Z & -f_y Y/Z^2 \end{bmatrix} \quad (2 \times 3), \quad (36)$$

so that

$$\mathbf{J}_i = \mathbf{D}_i \begin{bmatrix} \mathbf{I}_3 & -\mathbf{R} [\mathbf{P}_{\text{mk}}^{(i)}]_{\times} \end{bmatrix}, \quad \mathbf{J} = \begin{bmatrix} \mathbf{J}_1 \\ \mathbf{J}_2 \\ \mathbf{J}_3 \\ \mathbf{J}_4 \end{bmatrix} \in \mathbb{R}^{8 \times 6}. \quad (37)$$

The first three columns of $\mathbf{J}$ are in pixels per metre and the last three in pixels per radian. The principal point does not enter $\mathbf{J}$.

Verification. The implementation (`reprojection_jacobian`) was compared with centred finite differences computed independently of the project code: the projection was rewritten from scratch and the rotation was perturbed with SciPy rather than with the project's exponential map. At the baseline pose and at six viewpoints of the arc, including a camera with $f_x \neq f_y$ and a shifted principal point, the two agree to a relative accuracy of $2 \cdot 10^{-11}$.

4.2 Singular Values and Observability

The singular value decomposition

$$\mathbf{J} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top = \sum_{k=1}^6 \sigma_k \mathbf{u}_k \mathbf{v}_k^\top, \quad \sigma_1 \geq \dots \geq \sigma_6, \quad (38)$$

satisfies $\mathbf{J}\mathbf{v}_k = \sigma_k \mathbf{u}_k$. Moving the pose by a small amount ε along the right singular vector $\mathbf{v}_k$ changes the residuals by $\varepsilon \sigma_k$ in norm. A large σ_k therefore marks a direction of the pose that is clearly visible in the image, and a small one a direction along which the pose can move while the corners barely do. The smallest singular value $\sigma_{\min} = \sigma_6$ measures how poorly the worst direction is observed, $\mathbf{v}_6$ identifies that direction, and the condition number $\kappa = \sigma_1/\sigma_6$ compares the best and worst observed directions.

From singular values to uncertainty. Let the observed pixels carry independent noise $\mathbf{n}$ with $\text{Cov}(\mathbf{n}) = \sigma_{px}^2 \mathbf{I}$. Linearizing the residuals around the true pose, $\mathbf{e}(\delta\boldsymbol{\theta}) \approx \mathbf{J}\delta\boldsymbol{\theta} - \mathbf{n}$, the minimizer of E is the linear least-squares solution

$$\delta\boldsymbol{\theta} = (\mathbf{J}^\top \mathbf{J})^{-1} \mathbf{J}^\top \mathbf{n}, \quad (39)$$

whose covariance is

$$\text{Cov}(\delta\boldsymbol{\theta}) = \sigma_{px}^2 (\mathbf{J}^\top \mathbf{J})^{-1} = \sigma_{px}^2 \sum_{k=1}^6 \frac{\mathbf{v}_k \mathbf{v}_k^\top}{\sigma_k^2}. \quad (40)$$

The estimate thus scatters along each $\mathbf{v}_k$ with standard deviation σ_{px}/σ_k : the weakest direction carries the largest error. The prediction describes the minimizer of E ; Section 4.6 returns to this point.

4.3 Units and the Dimensionless Jacobian

The six parameters do not share a unit, so the singular values of $\mathbf{J}$ mix pixels per metre with pixels per radian, and the condition number depends on that choice. Measuring translation in millimetres instead of metres moves the condition number at a 30° view from 55 to 652, although nothing physical has changed. More tellingly, a 10 cm marker at 0.5 m and a 20 cm marker at 1 m produce exactly the same image, yet in SI units they have condition numbers 200 and 100 head-on.

We therefore measure translation in units of the camera–marker distance $Z = \|\mathbf{t}\|$ and work with the dimensionless Jacobian

$$\mathbf{J}_n = \mathbf{J} \mathbf{S}, \quad \mathbf{S} = \text{diag}(Z, Z, Z, 1, 1, 1). \quad (41)$$

A translation of one unit then displaces the marker by about one radian as seen from the camera, so all six parameters are dimensionless and comparable. The singular values of $\mathbf{J}_n$ are in pixels and its condition number κ is a pure number: identical images give identical values, and κ does not depend on the focal length. The rotation columns are unaffected. From here on, σ_k , $\mathbf{v}_k$ and κ refer to $\mathbf{J}_n$; the covariances of (40) remain in metres and radians.

4.4 The Fronto-Parallel Case in Closed Form

At a fronto-parallel view the Jacobian can be analysed by hand. Let $f_x = f_y = f$, let the marker face the camera at distance Z on the optical axis, $\mathbf{R} = \text{diag}(1, -1, -1)$ and $\mathbf{t} = (0, 0, Z)^\top$, and let the corners be $\mathbf{P}_{mk} = (x, y, 0)^\top$ with $x, y \in \{\pm h\}$ and $h = L/2$. This is the 0° pose of the

camera arc. All four corners lie at depth Z , and (37) gives, for each corner and in the order (34), the two rows

$$\begin{aligned} u\text{-row: } & \frac{f}{Z} \left(1, 0, -\frac{x}{Z}, \frac{xy}{Z}, -\frac{x^2}{Z}, -y \right), \\ v\text{-row: } & \frac{f}{Z} \left(0, 1, \frac{y}{Z}, -\frac{y^2}{Z}, \frac{xy}{Z}, -x \right). \end{aligned} \quad (42)$$

By the symmetry of the square, sums of odd powers of x and y vanish, and $\mathbf{J}^\top \mathbf{J}$ becomes block diagonal:

$$\frac{\mathbf{J}^\top \mathbf{J}}{(f/Z)^2} = \underbrace{\begin{bmatrix} 4 & -4s \\ -4s & 8s^2 \end{bmatrix}}_{(t_x, \omega_y)} \oplus \underbrace{\begin{bmatrix} 4 & -4s \\ -4s & 8s^2 \end{bmatrix}}_{(t_y, \omega_x)} \oplus \underbrace{\begin{bmatrix} 8h^2 \\ Z^2 \end{bmatrix}}_{t_z} \oplus \underbrace{\begin{bmatrix} 8h^2 \end{bmatrix}}_{\omega_z}, \quad s = \frac{h^2}{Z}. \quad (43)$$

Each out-of-plane rotation is coupled to one sideways translation, while depth and in-plane spin decouple. Since the marker is small compared with its distance, $s \ll 1$, and to leading order the singular values of the dimensionless Jacobian (41) are

$$\sigma_{t_x} = \sigma_{t_y} \approx 2f, \quad \sigma_{t_z} = \sigma_{\omega_z} = \frac{2\sqrt{2}fh}{Z}, \quad \sigma_{\min} \approx \frac{2fh^2}{Z^2}, \quad \boxed{\kappa \approx \left(\frac{Z}{h}\right)^2} \quad (44)$$

The condition number head-on depends only on the marker's apparent half-size h/Z : *the conditioning degrades as the inverse square of how large the marker appears.*

Why the tilts are the weak directions. The eigenvectors of the small eigenvalues are the two out-of-plane tilts, each accompanied by a small sideways translation. Rows (42) show why. Since $x^2 = h^2$ at every corner, a tilt $\delta\omega_y$ moves all four corners horizontally by the same amount, $-fh^2/Z^2$ per radian; that uniform shift is reproduced exactly by a sideways translation and therefore cancels. What remains is the vertical motion, whose sign follows xy : two opposite corners move up and the other two down, a small keystone distortion of the square. Head-on, that keystone is the only first-order evidence of a tilt, and its size is of order h/Z smaller than the effect of translating the marker.

Table 4 compares (44) with the implementation for $f = 800$ px, $h = 0.05$ m and $Z = 0.5$ m. The small differences are the neglected terms of order s^2 .

Quantity	Closed form	Implementation
$\sigma_{t_x} = \sigma_{t_y}$	1600.0	1600.08
σ_{t_z}	226.274	226.274
σ_{ω_z}	226.274	226.274
$\sigma_{\min}$ (twice)	16.000	15.9992
κ	$(Z/h)^2 = 100$	100.010

Table 4: Fronto-parallel view: closed-form singular values of the dimensionless Jacobian against the implementation.

4.5 Conditioning Along the Camera Arc

Away from the fronto-parallel view the algebra no longer closes, and the Jacobian is evaluated numerically (`run_conditioning_sweep`) at the true pose of every viewpoint of the arc of Experiment 1: radius 0.5 m, azimuth 0° , one viewpoint per degree from 0° to 89° . Figure 2 shows the result and Table 5 lists representative values.

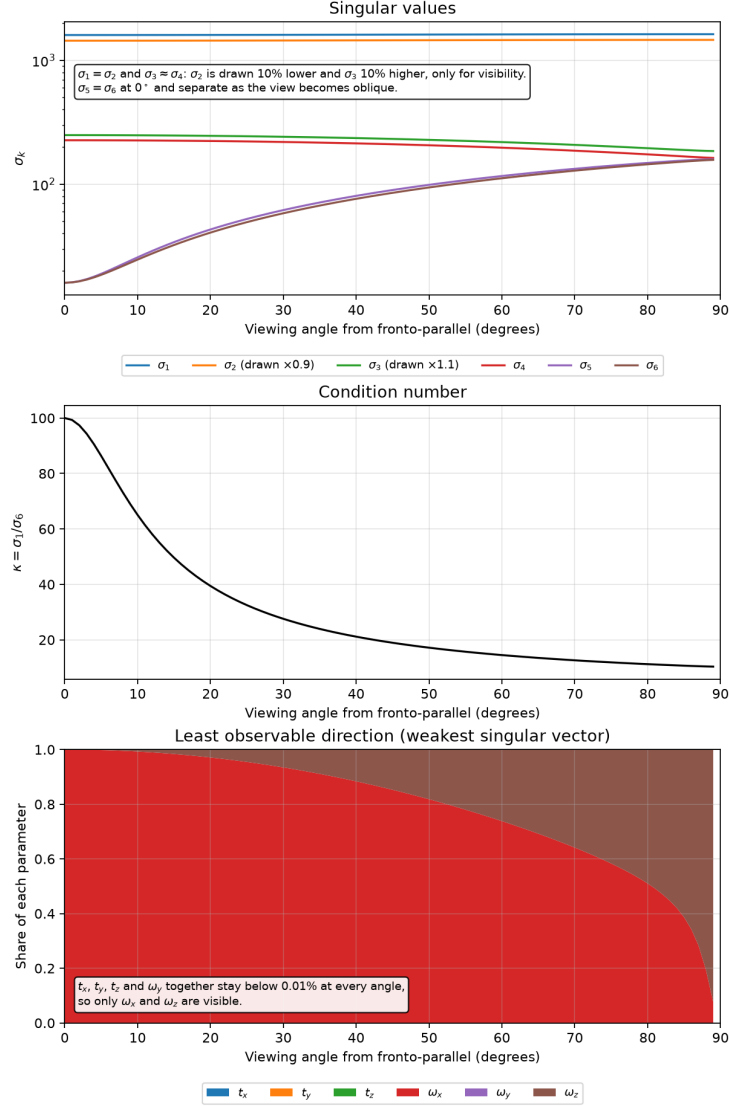


Figure 2: Conditioning of the dimensionless reprojection Jacobian along the camera arc. Top: the six singular values; $\sigma_1 = \sigma_2$ and $\sigma_3 \approx \sigma_4$, so σ_2 is drawn 10% lower and σ_3 10% higher for visibility. Middle: the condition number $\kappa = \sigma_1/\sigma_6$. Bottom: the share of each pose parameter in the least observable direction $\mathbf{v}_6$ (squared components); t_x , t_y , t_z and ω_y together stay below 0.01% at every angle. Generated by `experiment2.py`.

θ	σ_1	σ_2	σ_3	σ_4	σ_5	σ_6	κ	Weakest direction
0°	1600.1	1600.1	226.3	226.3	16.0	16.0	100.0	$\omega_x + 0.010 t_y$
5°	1600.3	1600.3	226.1	226.1	18.9	18.5	86.4	$\omega_x - 0.04 \omega_z$
15°	1601.7	1601.7	224.6	224.5	34.1	32.3	49.6	$\omega_x - 0.13 \omega_z$
30°	1606.2	1606.1	219.5	219.1	61.8	58.2	27.6	$\omega_x - 0.26 \omega_z$
45°	1612.3	1612.1	210.8	210.0	89.7	85.1	18.9	$\omega_x - 0.38 \omega_z$
60°	1618.4	1618.2	198.7	197.0	116.4	111.6	14.5	$\omega_x - 0.51 \omega_z$
75°	1622.9	1622.7	183.2	180.3	141.0	136.8	11.9	$\omega_x - 0.65 \omega_z$
89°	1624.5	1624.3	168.3	162.8	159.0	157.2	10.3	$\omega_z - 0.28 \omega_x$

Table 5: Singular values (pixels) and condition number of the dimensionless Jacobian along the arc, with the dominant components of the least observable direction.

Which motion each singular value belongs to. The singular vectors attach every singular value to one motion of the marker, and the geometry explains why they come in pairs.

- σ_1, σ_2 belong to the two sideways translations. Sliding the marker moves all four corners together; the two are equal because a square is unchanged by a 90° turn, and they stay nearly constant because they depend only on f and Z , both fixed along the arc.
- σ_3, σ_4 belong to depth and in-plane spin. Bringing the marker closer by one unit of Z zooms the image and moves each corner, at distance $r = h\sqrt{2}$ from the centre, outwards by fr/Z pixels; spinning it by one radian moves each corner along a circle by the same amount. Both decrease as the marker is foreshortened.
- σ_5, σ_6 belong to the two out-of-plane tilts. They are equal head-on by the same symmetry and small because a tilt then shows only as a keystone. The arc tilts the marker about its own Y axis, which breaks the symmetry and separates them.

Reading the condition number. A larger κ means a worse conditioned problem: the pose is then determined very unevenly, precisely in some directions and loosely in others. The fronto-parallel view is the worst case, $\kappa = 100$, and the conditioning improves steeply with obliquity: κ halves by 15° and reaches about 28 at 30° , after which the gains are small, down to 10 at 89° . Since σ_1 is essentially constant, the whole trend is carried by σ_6 , which grows tenfold from 16 to 157. The reason is geometric. Head-on, a small out-of-plane rotation moves the corners only along the line of sight and is visible solely through the keystone. Once the plane is oblique, the same rotation also moves the corners perpendicular to the line of sight, with opposite signs on opposite edges, which no translation can reproduce, so the evidence becomes first order in h/Z .

The least observable direction. It is always a rotation and never a translation: the three translations and ω_y together never exceed 0.01% of $\mathbf{v}_6$. Up to about 20° it is a pure out-of-plane tilt ω_x , the marker nodding about its horizontal axis, perpendicular to the axis about which the camera moves along the arc. As the view becomes oblique it acquires an in-plane spin component, and near 89° it becomes mostly ω_z : seen almost edge-on, spinning the marker in its own plane moves the corners along the line of sight, which the image hardly registers.

4.6 Monte Carlo Validation

4.6.1 The estimator

Equation (40) describes the *minimizer of E* , so the simulation must produce exactly that estimate. Since E is a sum of squared non-linear residuals, it is minimized with Gauss–Newton, which requires only the Jacobian (37). Starting from a pose, each iteration evaluates $\mathbf{e}$ and $\mathbf{J}$, computes the step

$$\delta\boldsymbol{\theta} = -(\mathbf{J}^\top \mathbf{J})^{-1} \mathbf{J}^\top \mathbf{e}, \quad (45)$$

which minimizes the linearized cost $\|\mathbf{e} + \mathbf{J} \delta\boldsymbol{\theta}\|^2$, and updates $\mathbf{t} \leftarrow \mathbf{t} + \delta\mathbf{t}$ and $\mathbf{R} \leftarrow \mathbf{R} \exp([\delta\boldsymbol{\omega}]_\times)$, until $\|\delta\boldsymbol{\theta}\| < 10^{-10}$. At the true pose, where $\mathbf{e} = -\mathbf{n}$, the step (45) is exactly (39): the prediction is one Gauss–Newton step, and the simulation tests whether the full, iterated minimization behaves as that first-order picture states.

Gauss–Newton is local, and for a planar marker E has two valleys, around the true pose and around its mirror. IPPE supplies one starting point in each: both candidates are refined and the one with the lower cost is kept (`estimate_pose`). The result does not depend on the starting point within a valley: on the same noisy data, Gauss–Newton started from the IPPE candidates and from the true pose reached the same estimate in all of 300 trials at 5° and at 30° , to within 10^{-11} degrees. Raw IPPE, by contrast, lies 2.7° from that minimizer at 5° (median), more than

the predicted scatter itself, so it could not be used in place of the minimizer.

4.6.2 Procedure

At each viewing angle $N = 2500$ trials are run. Each trial adds independent Gaussian noise with $\sigma_{px} = 0.5$ px to the eight corner coordinates, estimates the pose as above, and records its error in the coordinates of the Jacobian, $\delta \mathbf{t} = \hat{\mathbf{t}} - \mathbf{t}$ and $\delta \boldsymbol{\omega} = \log(\mathbf{R}^\top \hat{\mathbf{R}})$. A trial is counted as a *mirror flip* when the estimated rotation is closer to the mirror pose (IPPE’s second candidate on exact data) than to the true one; at 0° the two coincide and no flip is possible. Statistics are computed on the trials that remained in the valley of the true pose, since (40) describes a single minimum, and flips are reported separately. Three comparisons are made: the ratio of measured to predicted standard deviation for each parameter, whose statistical uncertainty over N trials gives a $\pm 2\sigma$ band of $\pm 2/\sqrt{2(N-1)} = \pm 2.8\%$; the measured scatter along $\mathbf{v}_6$ against σ_{px}/σ_6 ; and the fraction of flips. The viewing angles are dense near head-on, 0° , 0.5° , 1° , 2° , $\dots$, 8° , where the mirror can win, and every 5° from 10° to 85° .

4.6.3 Results

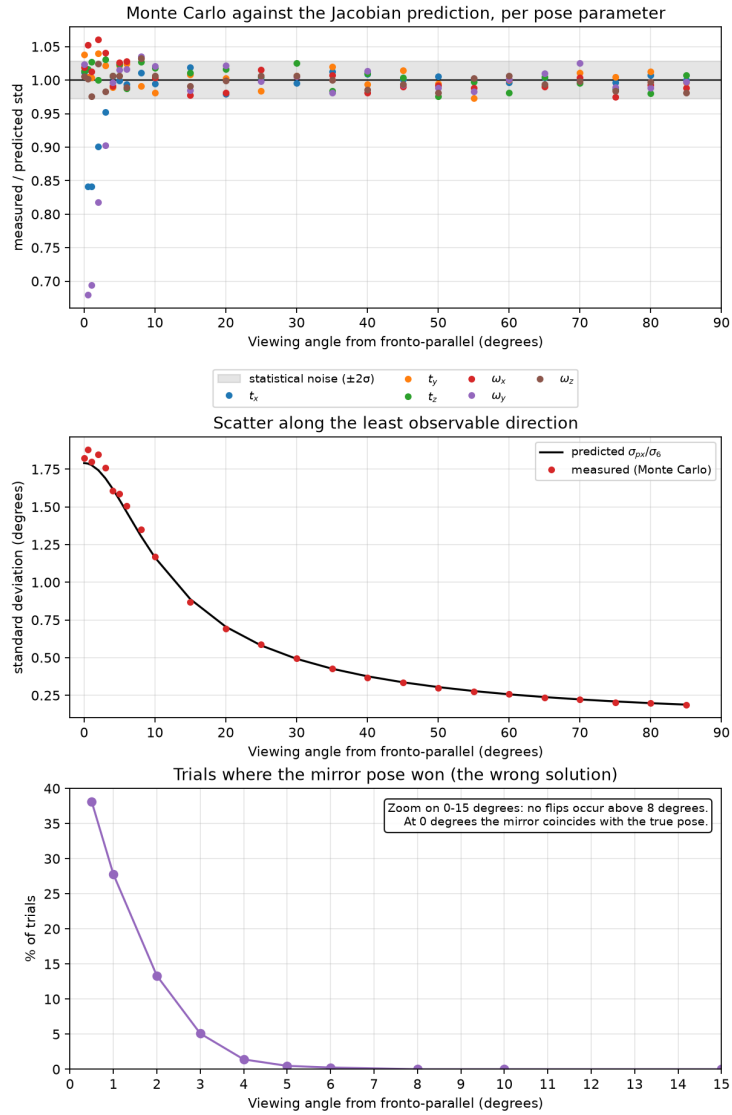


Figure 3: Monte Carlo validation, 2500 trials per angle, $\sigma_{px} = 0.5$ px. Top: ratio of measured to predicted standard deviation for each pose parameter; the grey band is the $\pm 2\sigma$ statistical uncertainty. Middle: standard deviation along the least observable direction, measured against the prediction σ_{px}/σ_6 . Bottom: percentage of trials in which the mirror pose was selected, on 0° – 15° . Generated by `experiment2.py`.

θ	Mirror flips	Ratio ω_y	Ratio t_x	$\mathbf{v}_6$ pred. ($^\circ$)	$\mathbf{v}_6$ meas. ($^\circ$)
0°	0	1.023	1.012	1.791	1.824
0.5°	953 (38%)	0.679	0.841	1.788	1.881
1°	694 (28%)	0.694	0.841	1.778	1.801
2°	333 (13%)	0.817	0.901	1.744	1.849
3°	127 (5%)	0.903	0.952	1.690	1.758
4°	35 (1.4%)	0.996	1.006	1.622	1.608
6°	6 (0.2%)	1.017	0.994	1.466	1.507
10°	0	1.021	0.995	1.164	1.168
30°	0	1.005	0.996	0.492	0.495
60°	0	1.001	0.996	0.257	0.258
85°	0	0.997	1.000	0.188	0.186

Table 6: Monte Carlo results at representative angles. Ratios are measured over predicted standard deviations. The other four parameters stay close to 1 at every angle; from 4° onwards all six lie within 3.6% of 1.

4.6.4 Interpretation

The prediction holds. At 0° and from 4° to 85° , the measured scatter agrees with $\sigma_{px}^2(\mathbf{J}^\top \mathbf{J})^{-1}$ within statistical noise, both parameter by parameter and along the least observable direction, whose standard deviation falls from 1.8° head-on to 0.19° at 85° . The error also concentrates where the conditioning analysis placed it: in the out-of-plane rotations, while the sideways translations scatter by about 0.2 mm at every angle.

Where it stops holding. Between 0.5° and 3° three effects appear together, with a single cause: the mirror pose lies within the noise.

- *Mirror flips.* The mirror is the true pose rotated by 2θ about the marker’s Y axis. Near head-on this is within the tilt scatter of about 1.8° , and noise frequently makes the mirror the better fit: 38% of the trials at 0.5° , tending to one half as $\theta \rightarrow 0$, and none from 8° onwards. In those trials the noisy corners genuinely fit the mirror better, so no estimator based on the four corners alone could avoid the error. This is the ambiguity of Experiment 1 measured under noise.
- *Truncated statistics.* Because the mirror lies along ω_y , the trials that flip are those with the largest ω_y errors, and excluding them makes the remaining distribution too narrow: ratios down to 0.68. The translation t_x , coupled to ω_y (Section 4.4), follows. A ratio below one therefore signals that the single-minimum prediction no longer applies, not that ω_y is better observed.
- *A slightly larger scatter along $\mathbf{v}_6$,* by 3–6%: with two minima this close, E is flatter than the quadratic bowl assumed by the first-order analysis.

At exactly 0° the two valleys merge into one, and the prediction holds again.

4.7 Discussion

Fronto-parallel is the worst case. Contrary to the intuition that looking straight at a marker is the easy configuration, the fronto-parallel view is the least well conditioned. Its condition number is $(Z/h)^2$: doubling the distance, or halving the marker, makes it four times worse.

What is lost is orientation, not position. The least observable direction is always a rotation. With $\sigma_{px} = 0.5$ px, the out-of-plane tilt is uncertain by 1.8° head-on, 0.5° at 30° and 0.26° at 60° , while the sideways position is determined to a fraction of a millimetre throughout. A modest obliquity of 20° – 30° secures most of the improvement.

The first-order theory is validated. With the pose estimated as the minimizer of E , the Jacobian predicts the Monte Carlo scatter within statistical noise everywhere except in a narrow band near head-on.

The two experiments describe one phenomenon. Experiment 1 showed that the two IPPE candidates differ by the sign of a vector of length $\sin \theta$, (30), so that near head-on the image reveals how much the marker is tilted but hardly towards which side. Experiment 2 shows that the same out-of-plane tilt is the least observable direction of the least-squares problem, with a first-order signature of relative size h/Z only. The Monte Carlo joins the two: the prediction fails exactly in the band where the mirror solution lies within the noise and is selected in up to 38% of the trials.